

[Home](#)   [Developer Reference](#)   [Plugin Development](#)

# Plugin Development

This page describes how to write Python plugins for OrcaSlicer and how to add new plugin types in C++. Python plugins run inside an embedded CPython interpreter, so they can extend OrcaSlicer without recompiling the application.

This page is for two audiences:

1. **Plugin authors** - writing a new Python plugin, modifying an existing one, and debugging it during development.
2. **OrcaSlicer contributors** - adding a brand-new *plugin type* in C++ (a new contract that Python plugins can implement and that the app invokes at some point in its workflow).

The related plugin development pages cover adjacent topics:

- [API Reference](#) - the embedded `orca` module, registration, host access, UI helpers, and each capability module.
- [Plugin Audit Hook](#) - the CPython audit hook that prompts the user before plugin code performs a sensitive filesystem, network, or process-spawn operation, and remembers the answer. Anyone adding a new trampoline method must read it; every C++ to Python call opens an audit context.
- [Plugin System Overview](#) - the catalog/loader/cloud-subscription side of the system (discovery, install, update).

## Note

All file paths below are under `src/slic3r/plugin/` unless stated otherwise.

## Part 1: Python Plugin Development

### Where plugins live and how they are discovered

Plugins are loaded from two roots under the OrcaSlicer data directory ( `data_dir()` ):

| Root                                  | Purpose                                 |
|---------------------------------------|-----------------------------------------|
| <code>data_dir()/orca_plugins/</code> | locally installed / side-loaded plugins |

| Root                                                              | Purpose                  |
|-------------------------------------------------------------------|--------------------------|
| <code>data_dir()/orca_plugins/_subscribed/&lt;user_id&gt;/</code> | cloud-subscribed plugins |

Each plugin lives in **its own subdirectory** containing exactly one entry file: either a single `.py` file or a single `.whl` (wheel). Subdirectories whose name starts with `.` or `__` are ignored. Discovery is driven by `PluginCatalog` (scan) and `PluginLoader` (load); see `PluginCatalog.cpp` and `PluginLoader.cpp` ( `find_installed_plugin_entry` in `PythonFileUtils.cpp` decides which file in a folder is the entry point).

### Note

There are **no bundled example plugins in the repository**. The plugin snippets in this document are illustrative. They were written against the real bindings below, but are not copied from a shipped, verified plugin. Treat them as starting points and test them.

## Anatomy of a plugin

A plugin is packaged in **one of two forms**, and the entry file in its folder is what distinguishes them:

- **A single `.py` file** - the simplest form, covered first below. Metadata lives in a PEP 723 comment block at the top of the file.
- **A wheel ( `.whl` )** - a normal built Python package, for plugins that need multiple modules or compiled code. Metadata comes from the wheel's own files instead of a PEP 723 block. See [Wheel \( `.whl` \) plugins](#) below.

**One plugin, many capabilities.** A plugin is a *package* that registers one or more **capabilities**. Each capability is a single typed unit of functionality: a script you can run, a G-code post-processor, a printer agent, with its own display name. A plugin's *types* are derived from the capabilities it registers (they are descriptive tags, not a single fixed role), so one plugin can, for example, offer both a script capability and a post-processing capability at once.

**Both forms register the same way:** OrcaSlicer imports your code, instantiates the one **package class** you marked with `@orca.plugin`, and calls its `register_capabilities()` method to collect the capabilities it offers. Only the packaging and the metadata source differ.

A single-file ( `.py` ) plugin has three parts:

1. A **PEP 723 inline metadata block** (a special comment header) declaring identity and dependencies.
2. One or more **capability classes**, each subclassing a typed base exposed by the embedded `orca` module (a script, slicing-pipeline, or printer-agent capability) and implementing `get_name()` plus its entry method.
3. A **package class** decorated with `@orca.plugin` (subclassing `orca.base`) whose `register_capabilities()` method calls `orca.register_capability(...)` once per capability.

### 1. The metadata block (PEP 723)

OrcaSlicer reads identity and dependency metadata from a PEP 723 *inline script metadata* block: a comment block delimited by `# /// script` and `# ///`, with each content line prefixed by `#`. Identity fields live in a `[tool.orcaslicer.plugin]` table; dependency fields live at the TOML root. Parsing is implemented in `PythonFileUtils.cpp::parse_pep723_toml / read_python_plugin_metadata`.

```
# /// script
# requires-python = ">=3.12"
# dependencies = []
#
# [tool.orcaslicer.plugin]
# name = "Sample Plugin"
# description = "Appends a short build-environment note to the exported G-code."
# author = "Your Name"
# version = "1.0.0"
# ///
```

| Field                    | Location                              | Required    | Notes                                      |
|--------------------------|---------------------------------------|-------------|--------------------------------------------|
| <code>name</code>        | <code>[tool.orcaslicer.plugin]</code> | recommended | display name in the Plugins dialog         |
| <code>description</code> | <code>[tool.orcaslicer.plugin]</code> | recommended | shown in the plugin <b>Description</b> tab |
| <code>author</code>      | <code>[tool.orcaslicer.plugin]</code> | optional    |                                            |
| <code>version</code>     | <code>[tool.orcaslicer.plugin]</code> | recommended |                                            |

| Field                        | Location  | Required | Notes                                                                        |
|------------------------------|-----------|----------|------------------------------------------------------------------------------|
| <code>dependencies</code>    | TOML root | optional | array of pip requirements (see <a href="#">Dependencies</a> )                |
| <code>requires-python</code> | TOML root | optional | <b>read but not stored or enforced</b> against the bundled interpreter today |

### Note

The metadata block no longer declares a `type`. A plugin's type(s) are derived from the capability classes it registers (each capability's `get_type()`), so the same metadata block is used whether the plugin offers one capability or several. The PEP 723 parser (`parse_pep723_toml`) reads only `name`, `description`, `author`, `version`, `requires-python`, and `dependencies`; any other key (including a stray `type = ...`) is ignored. The same applies to `.whl` plugins: identity comes from the wheel's `METADATA`, and the served types come from the registered capabilities.

## 2. The plugin API

The detailed API reference is split into [Registry](#), [Host](#), [Host UI](#), [Script](#), [Slicing Pipeline](#), and [Printer Agent](#). The summary below covers the main symbols used when writing a plugin.

The interpreter exposes a single embedded module named `orca` (`PYBIND11_EMBEDDED_MODULE(orca, ...)` in `PythonPluginBridge.cpp`). It provides the capability base classes, the package base, and capability registration, plus the `orca.host` submodule for read-only access to the live slicer model graph, presets, and mesh geometry (see [The `orca.host` module](#)). It contains:

| Symbol                       | Kind | Members / purpose                                                                                                                                                                                                                         |
|------------------------------|------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>orca.PluginType</code> | enum | <code>PrinterConnection</code> , <code>Automation</code> , <code>Analysis</code> , <code>Importer</code> , <code>Exporter</code> , <code>Visualization</code> , <code>Script</code> , <code>SlicingPipeline</code> , <code>Unknown</code> |

| Symbol                                     | Kind      | Members / purpose                                                                                                              |
|--------------------------------------------|-----------|--------------------------------------------------------------------------------------------------------------------------------|
| <code>orca.PluginResult</code>             | enum      | <code>Success</code> , <code>Skipped</code> , <code>RecoverableError</code> , <code>FatalError</code>                          |
| <code>orca.PluginContext</code>            | class     | base context, field <code>orca_version: str</code>                                                                             |
| <code>orca.ExecutionResult</code>          | class     | fields <code>status</code> , <code>message</code> , <code>data</code> ; factories below                                        |
| <code>orca.PythonPluginBase</code>         | class     | the root <b>capability</b> base; subclasses must implement <code>get_name()</code>                                             |
| <code>orca.base</code>                     | class     | the <b>package</b> base; subclass it and override <code>register_capabilities()</code>                                         |
| <code>orca.plugin</code>                   | decorator | marks the single package class for the file (exactly one per file)                                                             |
| <code>orca.register_capability(cls)</code> | function  | register one capability class; call it inside <code>register_capabilities()</code>                                             |
| <code>orca.slicing</code>                  | submodule | <b>Slicing Pipeline</b> : <code>Step</code> , <code>SlicingPipelineContext</code> , <code>SlicingPipelineCapabilityBase</code> |
| <code>orca.script</code>                   | submodule | <code>ScriptPluginCapabilityBase</code>                                                                                        |
| <code>orca.printer_agent</code>            | submodule | <code>PrinterAgentBase</code> and its data types                                                                               |
| <code>orca.host</code>                     | submodule | read-only host access: live <code>Model</code> graph, presets/bundle, and zero-copy mesh geometry                              |

`ExecutionResult` is how a plugin reports the outcome of a run:

```
orca.ExecutionResult.success(message="", data="")
orca.ExecutionResult.skipped(message="")
orca.ExecutionResult.failure(status, message, data="") # status is an
orca.PluginResult
```

- `status` - an `orca.PluginResult`.
- `message` - human-readable text; this is what surfaces in error/result dialogs.
- `data` - a free-form string whose meaning is defined by the plugin/workflow (not interpreted by the framework).

## Capability configuration

Every capability has a JSON configuration, regardless of its capability type. The **Config** tab in the Plugins dialog uses the host's JSON editor unless the capability supplies a custom HTML UI. The global configuration is stored under `data_dir()/orca_plugins/config.json` and is keyed by the full capability identity (plugin key, capability type, and capability name).

The configuration methods are available on `orca.PythonPluginBase`:

| Method                                        | Purpose                                                                                                                                                                                                                |
|-----------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>has_config_ui()</code> -<br>> bool      | Return <code>True</code> to use the custom UI returned by <code>get_config_ui()</code> instead of the JSON editor.                                                                                                     |
| <code>get_config_ui()</code> -<br>> str       | Return a self-contained HTML string. An empty or failing result falls back to the JSON editor.                                                                                                                         |
| <code>get_default_config()</code> -> str      | Override the method to return a Python <code>dict</code> ; OrcaSlicer exposes the resulting JSON as a string and uses it when the user restores global defaults. Without an override, the default is <code>{}</code> . |
| <code>get_config()</code> -> str              | Return this capability's effective stored configuration as a JSON string. It is <code>{}</code> when no configuration has been saved.                                                                                  |
| <code>get_config_version()</code> -> str      | Return the plugin version that last saved the configuration. Use this to detect and migrate an older schema.                                                                                                           |
| <code>save_config(config: str)</code> -> bool | Persist a JSON string for this capability. Use <code>json.dumps(config_dict)</code> and check the return value.                                                                                                        |

Example:

```
import json
import orca

class ConfigurableScript(orca.script.ScriptPluginCapabilityBase):
    def get_name(self):
        return "Configurable Script"
```

```

def get_default_config(self):
    return {"greeting": "Hello", "repeat": 1}

def execute(self):
    config = json.loads(self.get_config())
    greeting = config.get("greeting", "Hello")
    return orca.ExecutionResult.success(greeting * int(config.get("repeat",
1)))

def migrate_config_if_needed(self):
    version = self.get_config_version()
    config = json.loads(self.get_config())
    # Validate or migrate config here, then write it back with save_config().
    return version, config

```

`save_config()` supplies the plugin identity and current version automatically; a plugin cannot write another capability's configuration through this API. Preset-specific overrides are edited from a preset's plugin configuration dialog and are stored inside the preset. When an override is present, `get_config()` returns that override; otherwise it returns the global configuration.

For a custom UI, return a complete page from `get_config_ui()` and set `has_config_ui()` to `True`:

```

def has_config_ui(self):
    return True

def get_config_ui(self):
    return """<button id='save'>Save</button>
<script>
    const config = window.orca.getConfig();
    document.getElementById('save').onclick = () =>
        window.orca.saveConfig({enabled: true});
</script>"""

```

The page runs in a sandboxed frame. It can use only the injected `window.orca.getConfig()` and `window.orca.saveConfig(value)` bridge for configuration; keep the page self-contained and do not assume same-origin access to the host.

### The `orca.host` module: read-only host access

`orca.host` (bound in `PluginHostApi.cpp`) gives plugins **read-only** access to the running slicer. It is intended for analysis, reporting, and export plugins; nothing here mutates the model. **Script plugins run on the main/UI thread**, so within one `execute()` the model cannot change under you; **slicing-pipeline and printer-agent plugins run on a background**

**thread** while the GUI keeps running. Either way, treat everything as a momentary snapshot and do not stash references across runs.

**Entry points** (each raises `RuntimeError` if called before the GUI/model is ready):

```
import orca
model = orca.host.model()      # the active Model
plater = orca.host.plater()    # the Plater
bundle = orca.host.preset_bundle() # presets (prints/printers/filaments/...)
```

**Model graph:** `Model.objects()` -> `ModelObject`; each object has `volumes()` / `volume(i)` (-> `ModelVolume`) and `instances()` / `instance(i)` (-> `ModelInstance`). Bounding boxes are a `host.BoundingBox` value type (`min` / `max` / `size` / `center` as `(x, y, z)` mm tuples, plus `radius` / `defined`).

**Mesh geometry:** `ModelVolume.mesh()` -> `host.TriangleMesh`

| Member                                                                                                                      | Returns                                                                 | Notes                                                        |
|-----------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------|--------------------------------------------------------------|
| <code>vertex_count()</code> /<br><code>triangle_count()</code><br>( <code>facets_count()</code> ) / <code>is_empty()</code> | <code>int</code> / <code>bool</code>                                    | numpy-free                                                   |
| <code>vertex(i)</code> / <code>triangle(i)</code>                                                                           | <code>(x, y, z)</code> / <code>(a, b, c)</code> tuple                   | numpy-free, bounds-checked                                   |
| <code>vertices()</code>                                                                                                     | <code>(N, 3)</code> float32<br>ndarray                                  | <b>read-only, zero-copy</b> , requires numpy                 |
| <code>triangles()</code>                                                                                                    | <code>(M, 3)</code> int32<br>ndarray                                    | vertex indices; <b>read-only, zero-copy</b> , requires numpy |
| <code>face_normals()</code>                                                                                                 | <code>(M, 3)</code> float32<br>ndarray                                  | computed copy, requires numpy                                |
| <code>volume()</code> / <code>bounding_box()</code> /<br><code>is_manifold()</code>                                         | <code>float</code> /<br><code>BoundingBox</code> /<br><code>bool</code> | numpy-free                                                   |

Coordinates are **local** (the volume's own frame, in mm). The `vertices()` / `triangles()` arrays are zero-copy views into the live mesh and are marked read-only; writing to them raises

`ValueError`. Their lifetime is pinned to an immutable mesh snapshot, so they stay valid even if the volume's mesh is later replaced.

**Worked example** (declare numpy in the PEP 723 block so the bundled `uv` installs it):

```
# /// script
# dependencies = ["numpy"]
# ///
import orca, numpy as np

class MeshReport(orca.script.ScriptPluginCapabilityBase):
    def get_name(self): return "Mesh Report"
    def execute(self):
        model = orca.host.model()
        for obj in model.objects():
            for vol in obj.volumes():
                mesh = vol.mesh()
                V = np.asarray(mesh.vertices())          # (N, 3) float32, read-
only
                T = np.asarray(mesh.triangles())        # (M, 3) int32
                # World-space coordinates for the first instance (row-vector
convention):
                M = obj.instance(0).matrix() @ vol.matrix()    # 4x4 float64
                world = (np.c_[V.astype(np.float64), np.ones(len(V))] @ M.T)[:
:3]

                print(vol.name, V.shape, T.shape, world.min(0), world.max(0))
            return orca.ExecutionResult.success()

@orca.plugin
class MeshReportPlugin(orca.base):
    def register_capabilities(self):
        orca.register_capability(MeshReport)
```

### Note

If the instance is mirrored ( `instance.is_left_handed()` is `True`, i.e. `det(M) < 0` ), flip triangle winding / negate face normals when computing outward-facing normals in world space.

**numpy requirement:** `vertices()`, `triangles()`, `face_normals()`, and the `matrix()` accessors on `ModelVolume` / `ModelInstance` require numpy and raise a clear `ImportError` if it is not installed (declare `dependencies = ["numpy"]`). Everything else - counts, `vertex(i)` / `triangle(i)`, `volume()`, `bounding_box()`, `is_manifold()`, and the `offset` / `rotation` / `scaling_factor` / `mirror` tuple accessors - works without numpy.

**The `orca.host.ui` module: dialogs and interactive windows**

`orca.host.ui` lets a plugin show host-owned UI: a native message box, a native progress dialog, a modal HTML dialog, non-modal interactive windows, and a panel docked beside the 3D view. **A plugin must never import its own GUI toolkit** (PyQt/wxPython/tkinter): a `script` plugin shares the host's UI thread, so a second toolkit's event loop would clash with `wxWidgets`, and a `gcode / printer-agent` plugin runs off the main thread where toolkit calls would crash. These host calls run on the main thread for you and block the calling code until they return.

```
# Native message box -> returns "ok" | "cancel" | "yes" | "no"
choice = orca.host.ui.message("Export finished. Open the folder?",
                              title="My Plugin", buttons="yes_no",
                              icon="question")

# Modal HTML dialog -> returns the orca.submit() payload (dict), or None if
dismissed
result = orca.host.ui.show_dialog(html="<h2>Hello</h2> ...", title="Report",
                                  width=820, height=600)

# Non-modal, persistent, interactive window -> returns a UiWindow handle
win = orca.host.ui.create_window(html=PAGE, title="Panel",
                                 on_message=self.on_message,
                                 on_close=self.on_close)
win.post({"type": "data", "rows": [...]} # push a payload to the page
win.is_open() # bool
win.close()
```

message arguments: buttons is "ok"|"ok\_cancel"|"yes\_no"|"yes\_no\_cancel"; icon is "info"|"warning"|"error"|"question".

### Docked panels:

`create_dock_panel()` puts your page in a pane of the same dock area as the sidebar, instead of a separate window. The user can drag it to another edge, float it, resize it or close it. Opened again, it comes back where and how big it was when the window layout was last saved, which happens when a new project is started, not on exit. dock is "left", "right", "bottom" or "float"; width and height are in DIPs. It belongs to the Prepare and Preview tabs, and is hidden while the user is on another tab.

```
# Docked HTML panel -> returns a UiDockPanel handle
panel = orca.host.ui.create_dock_panel(html=PAGE, title="My Panel", width=320,
                                       height=480,
                                       on_message=self.on_message,
                                       on_close=self.on_close,
                                       dock="right")
panel.post({"type": "data", "rows": [...]} # push a payload to the page
panel.hide() # keep it open but out of the way
panel.show()
```

```

panel.is_open()                # True until closed; a hidden panel
is still open
panel.close()                  # fires on_close

```

The page talks to the plugin exactly as in a window, except `orca.submit()`, which only applies to modal dialogs.

### Progress dialogs:

Use `create_progress_dialog()` for host-owned native progress. It returns a `ProgressDialog` handle and also works as a context manager, so `close()` is called on exit. The default style is `PD_APP_MODAL | PD_AUTO_HIDE`; add `PD_CAN_ABORT` if the user should be able to cancel. `maximum` defaults to `100` (values `<= 0` are treated as `100`).

For script plugins, put the dialog inside `execute(self)` and update it between chunks of work. Do not create the dialog and then run one long uninterrupted operation such as a single `time.sleep(...)` or blocking network call; the dialog only gets useful repaint/cancel checkpoints when you call `update()` or `pulse()`.

```

style = (orca.host.ui.PD_APP_MODAL |
         orca.host.ui.PD_AUTO_HIDE |
         orca.host.ui.PD_CAN_ABORT |
         orca.host.ui.PD_ELAPSED_TIME |
         orca.host.ui.PD_REMAINING_TIME)

with orca.host.ui.create_progress_dialog("My Plugin",
                                       "Preparing...",
                                       maximum=len(items),
                                       style=style) as progress:
    for index, item in enumerate(items, start=1):
        process(item)

        # update() returns False if the dialog was closed or cancelled.
        if not progress.update(index, f"Processed {index}/{len(items)}"):
            return orca.ExecutionResult.skipped("Cancelled by user")

```

For indeterminate work, pulse the dialog instead of setting a numeric value:

```

with orca.host.ui.create_progress_dialog("My Plugin",
                                       "Waiting for printer...",
                                       style=orca.host.ui.PD_APP_MODAL |
                                       orca.host.ui.PD_CAN_ABORT) as
progress:
    while not finished():
        if not progress.pulse("Waiting for printer..."):
            return orca.ExecutionResult.skipped("Cancelled by user")
        wait_for_next_poll()

```

Handle methods:

| Python call                                                     | Effect                                                                                 |
|-----------------------------------------------------------------|----------------------------------------------------------------------------------------|
| <code>progress.update(value, message=" ")</code>                | set the determinate progress value; returns <code>False</code> if closed/cancelled     |
| <code>progress.pulse(message=" ")</code>                        | advance an indeterminate progress step; returns <code>False</code> if closed/cancelled |
| <code>progress.start_pulse(interval_ms=100, message=" ")</code> | start timer-driven pulsing on the UI thread                                            |
| <code>progress.stop_pulse()</code>                              | stop timer-driven pulsing                                                              |
| <code>progress.close()</code>                                   | close the dialog                                                                       |
| <code>progress.is_open()</code>                                 | return whether the host still has the dialog registered                                |

Because `start_pulse()` has no return value, use explicit `update()` or `pulse()` calls at natural cancellation points if the dialog includes `PD_CAN_ABORT`.

The style constants exposed by `orca.host.ui` mirror `wxProgressDialog`: `PD_APP_MODAL`, `PD_AUTO_HIDE`, `PD_CAN_ABORT`, `PD_CAN_SKIP`, `PD_ELAPSED_TIME`, `PD_ESTIMATED_TIME`, and `PD_REMAINING_TIME`. `PD_CAN_SKIP` is available for style parity, but the current Python handle does not expose a separate "skip" state.

**The page talks back through** `window.orca` (injected automatically; the page supplies raw, self-contained HTML/CSS/JS):

| JS call                            | Effect                                                                             |
|------------------------------------|------------------------------------------------------------------------------------|
| <code>orca.postMessage(obj)</code> | deliver <code>obj</code> to the plugin's <code>on_message(obj)</code>              |
| <code>orca.onMessage(cb)</code>    | <code>cb(data)</code> runs for each <code>win.post(data)</code> (and modal pushes) |
| <code>orca.submit(obj)</code>      | (modal) close and return <code>obj</code> from <code>show_dialog</code>            |
| <code>orca.close()</code>          | close the dialog / window                                                          |

## Theming (automatic light/dark):

The host injects a stylesheet that matches OrcaSlicer's **current theme** (the active light/dark mode, fonts, background/foreground, accent and border colors) *before* your page renders. An unstyled page already looks native: `<body>`, `headings`, `button`, `input / select / textarea`, `table`, links and scrollbars get sensible themed defaults, and the theme is also exposed as CSS variables so you can match the rest of the UI:

| Variable                      | Meaning                                     |
|-------------------------------|---------------------------------------------|
| <code>--orca-bg</code>        | window/background color                     |
| <code>--orca-fg</code>        | primary text color                          |
| <code>--orca-muted</code>     | secondary / label text color                |
| <code>--orca-accent</code>    | accent color (buttons, links, focus)        |
| <code>--orca-accent-fg</code> | text color on the accent                    |
| <code>--orca-border</code>    | subtle border / separator / row-hover color |
| <code>--orca-font</code>      | UI font stack                               |

The injected rules use only low specificity and never `!important`, so **any CSS your page ships overrides them**. Prefer the variables (e.g. `border:1px solid var(--orca-border)`) over hardcoded colors so your dialog follows light *and* dark mode automatically. The UI sample `host_ui_panel.py` relies on this and uses no fixed colors.

## Threading & lifecycle:

- Host UI calls run on the main thread and **block the calling code** until they return (`message / show_dialog` when the dialog closes; `create_window / create_progress_dialog` as soon as the window/dialog is shown; progress updates after the host applies them). From a `script` plugin, already on the UI thread, they run inline; from a background-thread plugin (`gcode / printer-agent`) they marshal to the main thread first.
- `on_message(data)` runs on the **UI thread**; keep it quick; offload heavy work to a `threading.Thread` and push results back with `win.post(...)`.
- A **modal** dialog (`show_dialog`) fits a one-shot `execute()`. A **persistent** panel (`create_window` or `create_dock_panel`) is best opened from `on_load()` so it lives for the

plugin's lifetime; the host closes a plugin's windows and panels automatically when it is unloaded/reloaded or the app exits, without calling `on_close`.

- Content is loaded as raw HTML; prefer **self-contained** pages (inline CSS/JS). There is no CSP and developer tools are disabled.

See `host_ui_panel.py` for a non-modal interactive panel that browses the whole `orca.host` read-only API.

### 3. Registration

Registration has two parts, both resolved at **module import / load time**.

**Capabilities** - each capability is a class that subclasses a typed base (see [Capability types and entry points](#)) and implements `get_name(self) -> str`. The name is how the capability appears in the UI and how presets refer to it, so it must be **unique within the plugin** and **must not contain a `;`**. That character is reserved as a separator in preset references, and a `;` in a capability name fails the load.

**The package** - exactly one class per file is decorated with `@orca.plugin` and subclasses `orca.base`. Its `register_capabilities(self)` method calls `orca.register_capability(Cls)` once for each capability class you want to expose:

```
@orca.plugin
class SamplePlugin(orca.base):
    def register_capabilities(self):
        orca.register_capability(GCodeBenchmark)
        orca.register_capability(EnvironmentReport)
```

OrcaSlicer instantiates the package class (it must be callable as `SamplePlugin()` with no arguments), calls `register_capabilities()`, then instantiates each registered capability.

Rules enforced when a plugin loads (most in `PythonPluginBridge.cpp`):

- The `@orca.plugin` class **must** subclass `orca.base`, and there must be **exactly one** per file. A second `@orca.plugin` fails the load.
- Each class passed to `orca.register_capability` must subclass a capability base (ultimately `orca.PythonPluginBase`); otherwise it raises `value_error`.
- Every capability must resolve `get_name()` (checked in the bridge); the loader (`PluginLoader.cpp`) additionally rejects the plugin if the resulting `(type, name)` pair is not unique across it.
- A capability class you never pass to `register_capability` is **invisible** to OrcaSlicer, even if it is defined in the file.

## Wheel ( `.whl` ) plugins

For anything beyond a single file, such as multiple modules, packaged resources, or compiled extensions, ship a standard Python **wheel** as the plugin folder's entry file. The plugin *code* is identical to the `.py` case: somewhere in the importable package's top-level code (typically its `__init__.py`) you define your capability classes and the `@orca.plugin` package class that registers them. What changes is **where metadata comes from** and that the wheel is validated on install ( `read_wheel_plugin_metadata` in `PythonFileUtils.cpp` ).

Identity and dependencies are read from the wheel's `*.dist-info/` files instead of a PEP 723 block:

| Plugin field              | Wheel source                            |
|---------------------------|-----------------------------------------|
| <code>name</code>         | METADATA -> Name ( <b>required</b> )    |
| <code>version</code>      | METADATA -> Version ( <b>required</b> ) |
| <code>description</code>  | METADATA -> Summary                     |
| <code>author</code>       | METADATA -> Author                      |
| <code>dependencies</code> | METADATA -> Requires-Dist               |

As with `.py` plugins, the wheel does **not** declare a type. The plugin's served types come from the capabilities its `@orca.plugin` package class registers at load time.

Additional wheel rules enforced at install time:

- The wheel must contain exactly **one** `.dist-info` **directory** with `METADATA`, `WHEEL`, and `RECORD` present.
- **Platform/ABI compatibility is checked** from the `WHEEL` file's `Tag:` lines. Pure Python wheels ( `*-none-any` ) are accepted everywhere; platform-specific wheels must match the current interpreter's ABI tag and OS (see `PythonInterpreter::python_abi_tag()` ). Ship a pure-Python wheel unless you genuinely need a compiled extension.
- The importable entry package is chosen in priority order: core-metadata `Import-Name`, then `top_level.txt` (if it names a single package), then the normalized `Name`.

## Capability Types and Entry Points

Each typed base defines the method(s) OrcaSlicer will call and the type returned by `get_type()`. Every capability **must** implement `get_name(self) -> str`. Lifecycle hooks

`on_load()` / `on_unload()` are optional and available on every capability (defaults do nothing).

The API Reference keeps the per-module details in separate pages: [Script](#), [Slicing Pipeline](#), and [Printer Agent](#).

| Base class                                              | <code>get_type()</code><br>returns | Required methods                                                                                                   | Invoked by                                                          |
|---------------------------------------------------------|------------------------------------|--------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------|
| <code>orca.script.ScriptPluginCapabilityBase</code>     | <code>Script</code>                | <code>get_name()</code> ,<br><code>execute(self) -&gt; ExecutionResult</code>                                      | the <b>Plugins dialog</b> -> <b>Run</b> action                      |
| <code>orca.slicing.SlicingPipelineCapabilityBase</code> | <code>SlicingPipeline</code>       | <code>get_name()</code> ,<br><code>execute(self, ctx) -&gt; ExecutionResult</code>                                 | configured slicing steps and <b>G-code export / post-processing</b> |
| <code>orca.printer_agent.PrinterAgentBase</code>        | <code>PrinterConnection</code>     | <code>get_name()</code> + ~30 agent methods<br>( <code>get_agent_info</code> , <code>connect_printer</code> , ...) | the <b>network / printer-agent</b> layer on load                    |

### Note

`get_name()` is required; `get_type()` usually is not. Every capability must implement `get_name()`. It is pure virtual on the root base, and a missing override fails the load. The typed C++ bases already implement `get_type()` (e.g. `ScriptPluginCapability::get_type()` returns `Script`), so a subclass of a *typed* base does **not** need to override it. Only a capability that subclasses the **root** `orca.PythonPluginBase` directly must set its own `get_type()`.

### ⚠ Important

`ScriptPluginCapabilityBase.execute()` runs on the **main/UI thread**: live host handles are safe to read for the whole call and `orca.host.ui` dialogs open inline, but a slow `execute()` **freezes the UI**. Keep it quick; offload heavy work to your own `threading.Thread` (which must not touch the model) and surface results through a `create_window` panel.

`SlicingPipelineCapabilityBase` / `PrinterAgentBase` instead run on background (slicing / network) threads.

The slicing context ( `orca.slicing.SlicingPipelineContext` ) is passed to `execute` and exposes the live slicing graph during geometry steps. At `Step.psGCodePostProcess`, the graph fields are `None` and the context instead exposes the working G-code file:

| Field                          | Meaning                                                                       |
|--------------------------------|-------------------------------------------------------------------------------|
| <code>orca_version</code>      | OrcaSlicer version string (inherited from <code>PluginContext</code> )        |
| <code>step</code>              | the pipeline step currently being executed                                    |
| <code>print</code>             | the live print graph, or <code>None</code> at <code>psGCodePostProcess</code> |
| <code>object</code>            | the current print object for object-scoped steps, or <code>None</code>        |
| <code>gcode_path</code>        | the working G-code path, set at <code>psGCodePostProcess</code>               |
| <code>host</code>              | target host, when exporting to a network printer                              |
| <code>output_name</code>       | the output file name                                                          |
| <code>config_value(key)</code> | the resolved print configuration value, or <code>None</code> if absent        |
| <code>cancelled()</code>       | whether the active print has been cancelled                                   |

### ⚠ Important

Filesystem, network, and process access are audited. While `execute()` runs, a sensitive operation (a file read/write/create/delete, an HTTP/socket connection, spawning a process) prompts the user with a Yes/No dialog naming the target; a "Yes" is remembered for that plugin so the same target does not prompt again. An operation inside a pre-determined allowed folder — `data_dir()`, the read-only `resources_dir()`, or (at `psGCodePostProcess`) the current G-code file's folder — needs no prompt at all; a `secrets / cert / conf`-like path is blocked outright, with no prompt either. See [Plugin Audit Hook](#).

## Complete Examples

**Minimal script plugin:** one capability, runs from the Plugins dialog, no context.

```
# /// script
# [tool.orcaslicer.plugin]
# name = "Hello Script"
# description = "Smallest possible script plugin."
# author = "Your Name"
# version = "1.0.0"
# ///
import orca

class HelloScript(orca.script.ScriptPluginCapabilityBase):
    def get_name(self):
        return "Hello Script"

    def on_load(self):
        # Optional: runs once when the capability is loaded.
        pass

    def execute(self):
        return orca.ExecutionResult.success("Hello from a script plugin")

@orca.plugin
class HelloPlugin(orca.base):
    def register_capabilities(self):
        orca.register_capability(HelloScript)
```

**Multi-capability plugin:** one package that exposes a slicing-pipeline capability *and* a script capability.

```

# /// script
# [tool.orcaslicer.plugin]
# name = "Sample Plugin"
# description = "Demonstrates registering several capabilities from one plugin."
# author = "Your Name"
# version = "1.0.0"
# ///
import orca

class EnvironmentReport(orca.slicing.SlicingPipelineCapabilityBase):
    def get_name(self):
        return "Environment Report"

    def execute(self, ctx):
        # Post-process the exported file at the dedicated pipeline seam.
        if ctx.step != orca.slicing.Step.psGCodePostProcess:
            return orca.ExecutionResult.success()
        try:
            with open(ctx.gcode_path, "a", encoding="utf-8") as f:
                f.write(f"\n; processed by Environment Report for
{ctx.output_name}\n")
        except Exception as exc:
            return orca.ExecutionResult.failure(
                orca.PluginResult.RecoverableError,
                f"could not append report: {exc}")
        return orca.ExecutionResult.success("report appended")

class GCodeBenchmark(orca.script.ScriptPluginCapabilityBase):
    def get_name(self):
        return "G-code Benchmark"

    def execute(self):
        return orca.ExecutionResult.success("benchmark complete")

@orca.plugin
class SamplePlugin(orca.base):
    def register_capabilities(self):
        orca.register_capability(EnvironmentReport)
        orca.register_capability(GCodeBenchmark)

```

For a copy-pasteable starter that registers a script, a post-processing, and a printer-agent capability in one package, see [multi\\_capability\\_skeleton.py](#).

### Note

When a capability is chosen for a setting (for example a post-processing capability), its `get_name()` is what the preset stores. The full reference saved alongside it is `<plugin_name>;<cloud_uuid>;<capability_name>`, which is why a capability name may not contain `;`. See [Plugin references in presets](#) for how this is used to restore missing plugins.

## Dependencies

List third-party requirements in the PEP 723 root `dependencies` array. On install, OrcaSlicer resolves them with a bundled `uv` into the plugin's environment (`PluginLoader.cpp`):

```
# dependencies = ["requests==2.32.3", "humanize"]
```

Keep dependencies minimal. Every dependency is code that runs under the same audit policy as your plugin and must be fetched at install time.

## Modifying an existing plugin

1. Locate its folder under `data_dir()/orca_plugins/<plugin>/` (or, for subscribed plugins, under `.../_subscribed/<user_id>/`).
2. Edit the `.py` entry file. If you change the metadata block, bump `version` so the change is visible in the Plugins dialog.
3. Reload (see the iteration workflow below). Note that plugin instances are captured at load time; a running OrcaSlicer will not pick up source edits until the plugin is reloaded or the app is restarted.

### Note

Editing a `.whl` plugin in place is not supported. Rebuild and reinstall the wheel.

## How Errors Are Surfaced

There are **three distinct error surfaces**. Knowing which one you are looking at tells you what kind of failure occurred.

1. **A message box:** a *runtime* failure of an explicit run. You get one when:

- your `execute()` **raised an exception** - it is caught at the C++ to Python trampoline boundary, the full traceback is logged, and the exception is rethrown and shown; or
- your `execute()` **returned a failure** (`ExecutionResult.failure(...)`, i.e. status `RecoverableError` / `FatalError`).

For **script** plugins the dialog title is "*Script Plugin Failed*" (or "*Script Plugin*" for a returned failure / success), and the body text is the exception message or your `ExecutionResult.message`. For **slicing-pipeline** plugins the failure is raised as a slicing error ("*Post-processing plugin <name> failed/raised...*") and surfaces through the normal slicing-error path. Source: `PluginsDialog.cpp` (`run_script_plugin`, `complete_with_error`) and `PostProcessor.cpp`.

**2. The plugin details / description area:** a *persistent* error state stored on the plugin descriptor, not a single run. When a plugin fails to load or has invalid metadata, the descriptor records an error (`set_error` / `normalized_error` in `PluginDescriptor.hpp`); for a metadata-invalid plugin **the error text replaces the description** shown in the dialog (`PluginsDialog.cpp`). This reflects *state* ("this plugin is currently broken") rather than the result of one execution.

**3. The Python log file:** the full traceback. `sys.stderr` is teed to:

```
data_dir()/log/python_<weekday>_<mon>_<day>_<HH>_<MM>_<SS>_<pid>.log
```

(`install_python_stderr_redirect` in `PythonInterpreter.cpp`.) This is the **only** place errors from background threads your plugin spawns will appear; those never cross back to C++ and never produce a dialog. C++-side context (load/discovery messages) goes to the main session log via Boost.

#### How to act on each:

- **Message box** -> read the message line, then open the `python_*.log` for the file/line of the traceback. Dialogs show only the message, not the stack.
- **Details-area / Diagnostics error** -> the plugin didn't load; usually a registration problem (a capability that doesn't subclass a typed base, a missing `get_name()`, a duplicate capability name, or no `@orca.plugin` package class) or an import error. Fix it, then reload.
- **Anything blocked with a `PermissionError`** -> the user answered "No" to the audit hook's prompt for that operation (or it was auto-denied). See the *Debugging* section of [Plugin Audit Hook](#) and the `[AUDIT BLOCKED]` log line.

**Prefer returning a result over raising** for failures you anticipate:

```
ExecutionResult.failure(orca.PluginResult.RecoverableError, "clear user-facing
```

`reason")` gives the user a clean message. Raise for genuine bugs; you'll get a full traceback in the log to debug from.

## Testing and Iterating During Development

A practical loop:

1. **Edit** the plugin source in its `orca_plugins/<plugin>/` folder.
2. **Reload** - reopen the Plugins dialog / re-trigger discovery, or restart OrcaSlicer if in doubt (instances are captured at load time).
3. **Run** - for a script plugin use the Plugins dialog **Run** action; for a slicing-pipeline plugin run a slice/export so the pipeline invokes it.
4. **Watch the log** - keep `data_dir()/log/python_*.log` open (e.g. `tail -f`). Tracebacks, `print()` output, and audit blocks all land there.
5. **Iterate**. Use `ExecutionResult` messages for expected outcomes; rely on the log for stack traces.

Tips:

- Confirm the plugin shows the right name and version in the Plugins dialog, and that **each capability you registered** appears (with the expected type) in its expandable capability list. A capability that is never passed to `orca.register_capability` will not appear.
- Develop against small, fast inputs; for slicing-pipeline plugins keep a tiny test model so each export cycle is quick.
- Remember that filesystem/network/process actions prompt on first use per plugin: a surprise `PermissionError` usually means an earlier prompt was answered "No" — or, for a path with `secret / cert / conf` in it, that it's categorically denied and was never prompted for at all. See [Plugin Audit Hook](#).

## Part 2: Adding a New Plugin Type in C++

This part is for OrcaSlicer contributors extending the plugin *framework* with a new contract, such as an "importer" capability type. The system has no per-type registry/switch for *instantiation*: a capability's Python class subclasses a typed base, the package's `register_capabilities()` registers it via `register_capability`, and the rest of the app reaches the loaded capability instance by `std::dynamic_pointer_cast<ConcreteType>` at the call site. So adding a type means: define a base + context + result, add a trampoline that forwards into Python through the audit macro, register pybind11 bindings, wire one call site, and add the files to the build.

Use the existing `slicingPipeline`, `script`, and `printerAgent` types under `src/slic3r/plugin/pluginTypes/` as references. `script` is the simplest, `slicingPipeline` shows a live graph context plus a per-call `audit_setup` callback, and `printerAgent` shows a wide multi-method interface.

## Step 1: Define the Plugin Contract

Create `pluginTypes/<type>/<Type>PluginCapability.hpp`. Subclass `PluginCapabilityInterface`, hardcode `get_type()` to your `PluginCapabilityType`, declare your pure-virtual entry method(s) and any context struct, and declare a static `RegisterBindings`. The current slicing-pipeline base (`pluginTypes/slicingPipeline/SlicingPipelinePluginCapability.hpp`) is the canonical context example:

```
#ifndef slic3r_SlicingPipelinePluginCapability_hpp_
#define slic3r_SlicingPipelinePluginCapability_hpp_

#include "../PythonPluginInterface.hpp"

namespace Slic3r {

struct SlicingPipelineContext {
    std::string orca_version;
    SlicingPipelineStepPlugin step;
    Print* print;
    const PrintObject* object;
    std::string gcode_path;
    std::string host;
    std::string output_name;
};

class SlicingPipelinePluginCapability : public PluginCapabilityInterface
{
public:
    PluginCapabilityType get_type() const override { return
    PluginCapabilityType::SlicingPipeline; }

    virtual ExecutionResult execute(SlicingPipelineContext& ctx) = 0;

    static void RegisterBindings(pybind11::module_ &module,
                                pybind11::enum_<PluginCapabilityType>
                                &pluginTypes);
};

} // namespace Slic3r

#endif /* slic3r_SlicingPipelinePluginCapability_hpp_ */
```

The shared building blocks come from `PythonPluginInterface.hpp`:

- `PluginCapabilityInterface` - `virtual std::string get_name() const = 0` (the capability's name, provided by Python), `virtual PluginCapabilityType get_type() const` (defaults to `Unknown`; typed bases override it), `virtual on_load() / on_unload()`, plus the C++-only audit identity (`set_audit_plugin_key`).
- `struct PluginContext { std::string orca_version; }` - derive your context from this.
- `struct ExecutionResult { PluginResult status; std::string message, data; }` with `static success / skipped / failure`.
- `enum class PluginCapabilityType { ... }` and the `plugin_capability_type_to_string / plugin_capability_type_from_string / plugin_capability_type_display_name` maps.

If your type needs a new `PluginCapabilityType` value, **add it to the enum and to all three maps** in `PythonPluginInterface.hpp`, choosing the string the maps translate. Reuse an existing value (e.g. `Automation`) if it fits.

## Step 2: Decide the API Surface

Decide exactly what the plugin must receive and return, and expose **only that**:

- **Inputs** go in the context struct (mirror `SlicingPipelineContext`). Keep it to data the plugin legitimately needs.
- **Outputs** should be an `ExecutionResult` (status + message + `data` string) unless your type genuinely needs richer return data. In that case, define and bind a small result struct.
- **Keep it minimal and stable.** The bindings are an API surface plugins depend on; removing or renaming a bound field/method breaks existing plugins. Add fields rather than repurpose them, and prefer the smallest interface that does the job.
- **Avoid exposing internal slicer/GUI types.** The current API deliberately exposes only plain data (strings, enums). Passing raw engine objects to plugins widens both the compatibility and the security surface.

## Step 3: Add the Trampoline

Create `pluginTypes/<type>/<Type>PluginCapabilityTrampoline.hpp`. Subclass `PyPluginCommonTrampoline<YourBase>` (which already provides the `get_name` and `on_load / on_unload` trampolines) and forward each virtual into Python via `ORCA_PY_OVERRIDE_AUDITED`. The slicing-pipeline trampoline (`pluginTypes/slicingPipeline/SlicingPipelinePluginCapabilityTrampoline.hpp`) follows this pattern:

```
#ifndef slic3r_SlicingPipelinePluginCapabilityTrampoline_hpp_
#define slic3r_SlicingPipelinePluginCapabilityTrampoline_hpp_

#include <boost/filesystem.hpp>
```

```

#include "../PyPluginTrampoline.hpp"
#include "../PluginAuditManager.hpp"
#include "SlicingPipelinePluginCapability.hpp"

namespace Slic3r {
class PySlicingPipelinePluginCapabilityTrampoline : public
PyPluginCommonTrampoline<SlicingPipelinePluginCapability>
{
public:
    using
PyPluginCommonTrampoline<SlicingPipelinePluginCapability>::PyPluginCommonTrampolin

    ExecutionResult execute(SlicingPipelineContext& ctx) override
    {
        ORCA_PY_OVERRIDE_AUDITED(
            [&] {
                // Only the export seam may write into the folder holding the
current G-code file.
                // The setup callback runs AFTER the context is constructed so the
scoped root
                // is not cleared by ScopedPluginAuditContext's constructor.

                if (!ctx.gcode_path.empty())

::Slic3r::PluginAuditManager::instance().add_scoped_allowed_root(
                    boost::filesystem::path(ctx.gcode_path).parent_path());
            },
            PYBIND11_OVERRIDE_PURE, ExecutionResult,
SlicingPipelinePluginCapability, execute, ctx);
    };
} // namespace Slic3r

#endif

```

The macros ( `PyPluginTrampoline.hpp` ) do two jobs at this single boundary: log + rethrow the Python traceback, and open the audit context.

```

ORCA_PY_OVERRIDE_AUDITED(audit_setup, override_macro, ret, base, name,
/*args...*/)

```

| Argument                 | Meaning                                                                                                                                                                  |
|--------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>audit_setup</code> | a lambda run <i>after</i> the audit context is opened; use it to<br><code>add_scoped_allowed_root(...)</code> if your call needs one. Pass <code>[] {}</code> if<br>none |

| Argument                          | Meaning                                                                                                                               |
|-----------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| <code>override_macro</code>       | pybind11's own <code>PYBIND11_OVERRIDE</code> (has a C++ fallback) or <code>PYBIND11_OVERRIDE_PURE</code> (pure virtual, no fallback) |
| <code>ret, base, name, ...</code> | the standard pybind11 override arguments                                                                                              |

### ⚠ Important

Every new trampoline method must go through `ORCA_PY_OVERRIDE_AUDITED` (or call `ORCA_PY_AUDIT_SCOPE()` directly if you need to hand-unpack the Python return value). There is no mode to choose — read [Plugin Audit Hook](#) to understand what gets prompted and persisted for the events your new type's Python surface can trigger.

## Step 4: Register the Python Bindings

Implement `RegisterBindings` in `pluginTypes/<type>/<Type>PluginCapability.cpp`: create a submodule, bind the context/result structs, and bind the base class with its trampoline. The slicing-pipeline implementation

(`pluginTypes/slicingPipeline/SlicingPipelinePluginCapability.cpp`) follows this shape:

```
void SlicingPipelinePluginCapability::RegisterBindings(pybind11::module_& module,
pybind11::enum_<PluginCapabilityType>& pluginTypes)
{
    (void) pluginTypes;

    auto slicing = module.def_submodule("slicing", "Slicing pipeline API");

    py::class_<SlicingPipelineContext>(slicing, "SlicingPipelineContext")
        .def(py::init<>())
        .def_readonly("step", &SlicingPipelineContext::step)
        .def_readonly("gcode_path", &SlicingPipelineContext::gcode_path)
        .def_readonly("host", &SlicingPipelineContext::host)
        .def_readonly("output_name", &SlicingPipelineContext::output_name);

    py::class_<SlicingPipelinePluginCapability, PluginCapabilityInterface,
PySlicingPipelinePluginCapabilityTrampoline,
std::shared_ptr<SlicingPipelinePluginCapability>>(slicing,
"SlicingPipelineCapabilityBase")
        .def(py::init<>())
        .def("get_type", &SlicingPipelinePluginCapability::get_type)
        .def("execute", &SlicingPipelinePluginCapability::execute);
}
```

```
}

```

The base class is bound as `SlicingPipelineCapabilityBase` (the name plugin authors subclass) and inherits `get_name` from the root `PythonPluginBase`, so you only bind the type-specific methods here. Then **call your `RegisterBindings` from `bind_python_api`** in `PythonPluginBridge.cpp`, next to the existing ones (look for the `// Make sure you register your bindings here` comment):

```
// Make sure you register your bindings here
SlicingPipelinePluginCapability::RegisterBindings(m, pluginTypes);
PrinterAgentPluginCapability::RegisterBindings(m, pluginTypes);
ScriptPluginCapability::RegisterBindings(m, pluginTypes);
PluginHostApi::RegisterBindings(m);
// YourTypeCapability::RegisterBindings(m, pluginTypes); // <-- add this

```

The shared `PluginCapabilityType` / `PluginResult` / `PluginContext` / `ExecutionResult` / `PythonPluginBase` bindings, the package base (`orca.base`), and the `@orca.plugin` / `orca.register_capability` entry points are already defined once in that same function. You only add your type-specific submodule.

### Step 5: Add Audit Hooks

Auditing is not optional. Each trampoline method you wrote in Step 3 already opens an audit context through `ORCA_PY_OVERRIDE_AUDITED`, so any filesystem/network/process operation your type's Python surface performs is automatically categorized, prompted, and (for most categories) persisted per plugin — see [Plugin Audit Hook](#) for the category table. If your type performs a sensitive operation the current event map doesn't cover, add the CPython event to `audit_event_categories` in `PluginAuditManager.cpp` rather than inventing a parallel check. Read the audit doc before finalizing your trampoline.

### Step 6: Hook the Type Into an OrcaSlicer Workflow

Nothing runs your plugin until some part of the app invokes it. Pick the invocation pattern that matches your type and model it on an existing one:

| Type                         | Where it's invoked                                        | Pattern                                                                                                                                                                                   |
|------------------------------|-----------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>slicingPipeline</code> | <code>Print.cpp</code> and <code>PostProcessor.cpp</code> | resolve the preset's capability refs, cast to <code>SlicingPipelinePluginCapability</code> , build <code>SlicingPipelineContext</code> , and call <code>execute(ctx)</code> under the GIL |

| Type         | Where it's invoked                               | Pattern                                                                                                                                                                                                                           |
|--------------|--------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| script       | PluginsDialog.cpp (Run action)                   | get_plugin_capability_by_name(...),<br>dynamic_pointer_cast<ScriptPluginCapability><br>(cap->instance), call execute()                                                                                                            |
| printerAgent | NetworkAgentFactory.cpp, wired<br>in GUI_App.cpp | register via subscribe_on_capability_load_callback<br>/ subscribe_on_capability_unload_callback; the<br>callback filters by capability.type ==<br>PluginCapabilityType::PrinterConnection, then<br>registers/deregisters an agent |

For your new type, add a call site (or an on-capability-load callback) that:

1. obtains a loaded capability (via `PluginLoader::get_plugin_capabilities_by_type(...)` or `get_plugin_capability_by_name(...)`) and does  
`std::dynamic_pointer_cast<YourTypeCapability>(cap->instance);`
2. on a successful cast, builds the context and invokes your entry method under the GIL;
3. if your type needs unload cleanup, add a case to the capability-teardown switch (keyed on `PluginCapabilityType`) in `PluginLoader.cpp`.

### ⚠ Important

Disabled / missing plugins must not change existing behavior. Every existing path is gated on a successful `dynamic_pointer_cast` (or a `type ==` check) and iterates only over installed/selected plugins, so when none of your type is installed the loop or callback simply finds nothing and does nothing. Follow the same pattern: never run unconditional work on behalf of a plugin type that isn't present.

## Step 7: Add the Files to the Build

List your new `.hpp` / `.cpp` files in `src/slic3r/CMakeLists.txt`, alongside the existing plugin-type sources (search for `plugin/pluginTypes/slicingPipeline/SlicingPipelinePluginCapability.cpp`; the block is around lines 615-623):

```
plugin/pluginTypes/<type>/<Type>PluginCapability.hpp
```



```
plugin/pluginTypes/<type>/<Type>PluginCapability.cpp
plugin/pluginTypes/<type>/<Type>PluginCapabilityTrampoline.hpp
```

## Recipe at a Glance

1. **Enum/maps** (if new type): add a `PluginCapabilityType` value + the three string maps in `PythonPluginInterface.hpp`.
2. **Contract**: `pluginTypes/<type>/<Type>PluginCapability.hpp` - base + context + result + static `RegisterBindings`.
3. **Trampoline**: `pluginTypes/<type>/<Type>PluginCapabilityTrampoline.hpp` - forward each virtual via `ORCA_PY_OVERRIDE_AUDITED`.
4. **Bindings**: `pluginTypes/<type>/<Type>PluginCapability.cpp` `RegisterBindings`, then call it from `bind_python_api` in `PythonPluginBridge.cpp`.
5. **Audit**: confirm the events your type's Python surface can trigger are categorized correctly per [Plugin Audit Hook](#).
6. **Workflow**: add a call site / on-load callback that casts and invokes; gate it so an absent type is a no-op.
7. **Build**: add the files to `src/slic3r/CMakeLists.txt`.

## Part 3: Testing and Verification

There is **no dedicated automated test suite for the Python plugin system today**.

Verification is primarily manual, with targeted Catch2 tests where the logic is pure C++.

### Manual Testing

- **Loading** - install/side-load a plugin into `data_dir()/orca_plugins/<name>/`, open the Plugins dialog, and confirm it appears with the correct name and version and that each registered capability is listed (with the expected type). A plugin that fails to load shows its error in the Diagnostics tab (see [How errors are surfaced](#)).
- **Execution** - script plugins: use the dialog **Run** action. Slicing-pipeline plugins: run a slice/export and confirm the plugin ran (e.g. its effect on the G-code, plus log output). Printer-agent plugins: verify the agent registers on load and deregisters on unload.
- **Error handling** - deliberately make the plugin (a) raise an exception and (b) return `ExecutionResult.failure(...)`; confirm the message box text, and that the full traceback appears in `data_dir()/log/python_*.log`. Confirm an invalid-metadata plugin surfaces its error in the details area rather than crashing.
- **Audit** - trigger a filesystem/network/process action, confirm the Yes/No dialog names the right target, and that answering **No** blocks it with a `PermissionError` and an `[AUDIT`

`BLOCKED]` log line while answering **Yes** lets it through and persists the grant so a repeat of the same action does not prompt again.

## Automated Tests Where Appropriate

Add **targeted Catch2 tests** (under `tests/`) for the pure-C++ pieces that don't need a running interpreter or GUI. For example:

- PEP 723 metadata parsing ( `parse_pep723_toml` / `read_python_plugin_metadata` in `PythonFileUtils.cpp` ): valid blocks, missing fields, malformed arrays.
- Capability reference parsing/serialization ( `parse_capability_ref` in `Config.cpp` ) - see `tests/libslc3r/test_config.cpp` and `tests/slic3rutils/test_plugin_capability_identifier.cpp` for local vs. cloud refs and malformed input.
- The allowed-root/denied-path logic ( `PluginAuditManager::check_open` , `is_inside_allowed_root` , `is_denied_path_keyword` ) that `tests/slic3rutils/test_plugin_audit.cpp` already covers: inside/outside roots, read-only roots, `..` traversal, read vs write, denied filenames and keywords. See [Plugin Audit Hook](#).
- Type-string round-trips ( `plugin_capability_type_from_string` / `plugin_capability_type_to_string` ).

Anything that requires the embedded interpreter, file installs, or GUI dialogs is currently best covered by the manual steps above.

## Cross-Platform and Regression Checks

- **Cross-platform** - the plugin code must build and run on Windows, macOS, and Linux. Be careful with path handling (audit target extraction canonicalizes paths via `PyOS_FSPath` ; keep using `std::filesystem` / the existing helpers), and with line endings in the PEP 723 parser (it already strips `\r` ).
- **No regressions** - changes to the framework must not alter behavior when no plugin of a given type is installed (Step 6). When touching the trampoline/audit headers, note that `PyPluginTrampoline.hpp` and `PluginAuditManager.hpp` are included by many translation units; a header-only change may need a clean rebuild of the affected targets to take effect (see the audit doc's *Debugging* section).
- **Backward compatibility** - don't rename or remove bound fields/methods or `PluginCapabilityType` values that existing plugins or installed profiles may depend on; add rather than repurpose.

## Key Files

| File                                                                                | Responsibility                                                                                                                                                                        |
|-------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>src/slic3r/plugin/PythonPluginInterface.hpp</code>                            | <code>PluginCapabilityType</code> , <code>PluginContext</code> , <code>PluginResult</code> , <code>ExecutionResult</code> , <code>PluginCapabilityInterface</code> , type-string maps |
| <code>src/slic3r/plugin/PythonPluginBridge.{hpp,cpp}</code>                         | the <code>orca</code> module ( <code>bind_python_api</code> ), <code>@orca.plugin / register_capability</code> , package + capability capture/instantiation                           |
| <code>src/slic3r/plugin/PyPluginPackage.hpp</code>                                  | the package base ( <code>orca.base</code> ) and its <code>register_capabilities</code>                                                                                                |
| <code>src/slic3r/plugin/PyPluginTrampoline.hpp</code>                               | the <code>ORCA_PY_*</code> trampoline macros (traceback logging + audit scope) and common trampolines                                                                                 |
| <code>src/slic3r/plugin/pluginTypes/&lt;type&gt;/</code>                            | per-type capability base ( <code>*PluginCapability.hpp/.cpp</code> ) and trampoline ( <code>*PluginCapabilityTrampoline.hpp</code> )                                                  |
| <code>src/slic3r/plugin/PluginDescriptor.hpp</code>                                 | per-plugin metadata + error state ( <code>set_error</code> , <code>normalized_error</code> , <code>is_metadata_valid</code> )                                                         |
| <code>src/slic3r/plugin/PythonFileUtils.cpp</code>                                  | PEP 723 / wheel metadata parsing, entry-file discovery                                                                                                                                |
| <code>src/slic3r/plugin/PluginCatalog.cpp</code> ,<br><code>PluginLoader.cpp</code> | discovery, install, load lifecycle, dependency install                                                                                                                                |
| <code>src/slic3r/plugin/PythonInterpreter.cpp</code>                                | interpreter init, audit-hook install, traceback formatting, <code>stderr</code> -> log file                                                                                           |
| <code>src/slic3r/GUI/PluginsDialog.cpp</code>                                       | Plugins dialog: details/error area, script <b>Run</b> , error dialogs                                                                                                                 |
| <code>src/slic3r/GUI/PostProcessor.cpp</code>                                       | resolves the preset's plugin refs and invokes post-processing (G-code) capabilities during export                                                                                     |
| <code>src/slic3r/CMakeLists.txt</code><br>(~609-623)                                | build list for plugin sources                                                                                                                                                         |

| File                              | Responsibility                                                  |
|-----------------------------------|-----------------------------------------------------------------|
| <a href="#">Plugin Audit Hook</a> | the audit hook: categories, prompting/persistence, extending it |